

The background features a dark blue, stylized illustration of a surfer riding a wave. The wave is depicted with glowing, digital-like lines and patterns, suggesting a high-tech or cybernetic theme. The surfer is shown in a dynamic, crouched position, riding the crest of the wave. In the upper left corner, there are two large, overlapping geometric shapes: a blue parallelogram and a light green parallelogram, both tilted at an angle.

MIMUW Surfers

Projekt zaliczeniowy JNP3

Agnieszka Suszko & Konrad Mocarski

Opis gry

- Gracz steruje biegącą postacią, której zadaniem jest przebiec jak najdalej (endless runner)
- Postać może poruszać się w obrębie 3 alejek/torów, z których na każdej pojawiają się przeszkody:
 - Barrierki, które można przeskoczyć
 - Barrierki, pod którymi można się prześlizgnąć
 - Pociągi - można na nie wejść
- Gdy gracz zderzy się z przeszkodą, przegrywa
- W trakcie gry, gracz może zdobywać monety oraz power upy





Build + grywalność

- Pełna grywalność na Windowsie
- Stan zwycięstwa/porażki:
 - Stanem zwycięstwa / celem gry jest uzyskanie jak najlepszego wyniku w przebytych dystansie i zdobytych monetach -> Gracz poczuje się zwycięzcą, jeśli przebije swój poprzedni wynik
 - Porażka w momencie zderzenia z przeszkodą (o tym za chwilę przy kolizjach)

Input

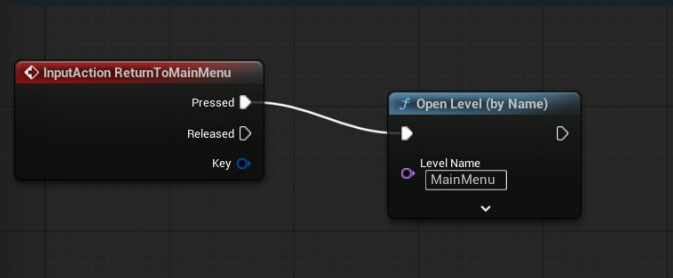
- Input mapping:
 - A - przejście na lewy tor
 - D - przejście na prawy tor
 - W / Spacja - skok
 - S - dodge
 - Esc - przejście do main menu w trakcie gry
- Binding w SetupPlayerInputComponent oraz w Blueprincie

```
// Called to bind functionality to input
void ASurferCharacter::SetupPlayerInputComponent(UInputComponent* PlayerInputComponent)
{
    Super::SetupPlayerInputComponent(PlayerInputComponent);

    PlayerInputComponent->BindAction("Jump", KeyEvent: IE_Pressed, Object: this, &ACharacter::Jump);
    PlayerInputComponent->BindAction("Jump", KeyEvent: IE_Released, Object: this, &ACharacter::StopJumping);

    // Lane switching
    PlayerInputComponent->BindAction("SwitchLaneLeft", KeyEvent: IE_Pressed, Object: this, &ASurferCharacter::SwitchLaneLeft);
    PlayerInputComponent->BindAction("SwitchLaneRight", KeyEvent: IE_Pressed, Object: this, &ASurferCharacter::SwitchLaneRight);

    // Dodge binding
    PlayerInputComponent->BindAction("Dodge", KeyEvent: IE_Pressed, Object: this, &ASurferCharacter::Dodge);
}
```



Wczytywanie scen

- Levele z różnym układem przeszkód, dziedziczące po klasie BaseLevel w c++
- Losowe spawnowanie monet na poziomie (o tym więcej w wybranej mechanice)
- Klasa C++ LevelSpawner
 - Max 10 leveli, usuwamy niepotrzebne
 - Gdy gracz przekroczy Trigger, w miejscu SpawnLocationBox najnowszego poziomu zostaje wylosowany i umieszczony nowy poziom
 - Możliwość podpięcia dowolnej liczby leveli w Blueprincie LevelSpawnera

```
void ALevelSpawnerLevelList::SpawnLevel(bool bFirst)
{
    SpawnLocation = FVector(1000.0f, 1000.0f, 1000.0f);
    SpawnRotation = FRotator(1000.0f, 1000.0f, 1000.0f);

    if (!bFirst && SpawnedLevels.Num() > 0)
    {
        ABaseLevel* LastLevel = SpawnedLevels.Last();
        SpawnLocation = LastLevel->GetSpawnLocation()->GetComponentTransform().GetTranslation();
    }

    // Safety check: ensure the array isn't empty to prevent crashing
    if (SpawnedLevels.Num() == 0)
    {
        UE_LOG(LogTemp, Warning, TEXT("No Levels set in the LevelSpawner Blueprint!"));
        return;
    }

    // Get a random index valid for the current array size
    RandomLevelIndex = FMath::RandRange(0, SpawnedLevels.Num() - 1);

    // Pick the class from the array
    TSubclassOf<ABaseLevel> SelectedLevelClass = Levels[RandomLevelIndex];

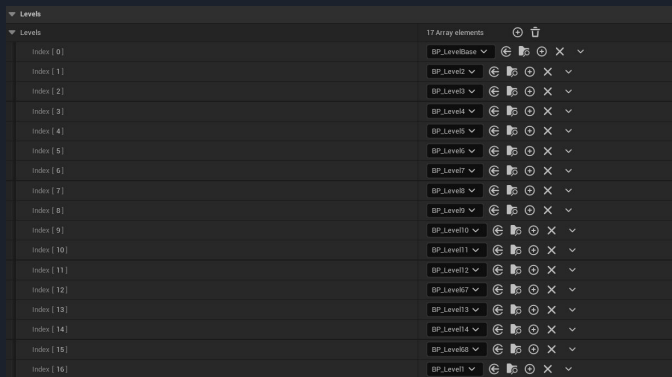
    ABaseLevel* NewLevel = nullptr;

    if (SelectedLevelClass)
    {
        NewLevel = GetWorld()->SpawnActor<ABaseLevel>(SelectedLevelClass, SpawnLocation, SpawnRotation, SpawnInfo);

        if (NewLevel)
        {
            if (NewLevel->SetTrigger())
            {
                NewLevel->SetTrigger()->OnComponentBeginOverlap.AddDynamic(this, &LevelSpawnerLevelList::OnOverlapBegin);
            }

            SpawnedLevels.Add(NewLevel);
        }
    }

    // Optimization: Destroy the old level to free up memory
    if (SpawnedLevels.Num() > 10)
    {
        ABaseLevel* OldLevel = SpawnedLevels[0];
        if (OldLevel)
        {
            OldLevel->Destroy(); // Actually remove the actor from the game world
        }
        SpawnedLevels.RemoveAt(0); // Remove pointer from list
    }
}
```



Detekcja kolizji

- Kolizja gracza z Triggerem (wczytywanie scen)
- Pociągi składają się z TrainBody, który blokuje kolizje (dzięki temu gracz może po nich chodzić) oraz CollisionBoxa z przodu (gracz przegrywa jeśli zderzy się z pociągiem od przodu).
 - Pociągi z rampą mają dezaktywowany CollisionBox i dodatkowo posiadają schody z przodu, które pozwalają graczowi dostać się na dach pociągu.
- Kolizja z barierką również powoduje przegraną



```
void ASurferCharacter::OnOverlapBegin(UPrimitiveComponent* OverlappedComp, AActor* OtherActor,
                                      UPrimitiveComponent* OtherOverlappedComponent, int32 OtherBodyIndex, bool bFromSw
{
    if (OtherActor != nullptr)
    {
        // Debug: Print what actor we're overlapping with
        UE_LOG(LogTemp, Warning, TEXT("Overlap with: %s"), *OtherActor->GetName());

        ASpike* Spike = Cast<ASpike>(OtherActor);
        ATrain* Train = Cast<ATrain>(OtherActor);

        UpdateCoins(OtherActor);

        if (Spike || Train)
        {
            UE_LOG(LogTemp, Warning, TEXT("Hit an obstacle! Restarting..."));

            // Play hit sound if assigned
            if (HitObstacleSound)
            {
                UGameplayStatics::PlaySoundAtLocation(this, HitObstacleSound, GetActorLocation());
            }

            if (ExplosionVFX)
            {
                UNiagaraFunctionLibrary::SpawnSystemAtLocation(
                    GetWorld(),
                    ExplosionVFX,
                    GetActorLocation(),
                    GetActorRotation()
                );
            }

            GetMesh()->Deactivate();
            GetMesh()->SetVisibility(false);
            CanMove = false;
        }
    }
}
```

Wybrana mechanika

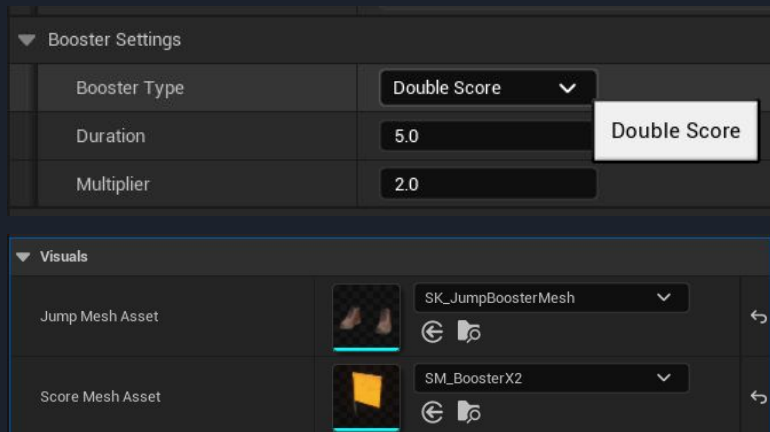
- Naszą wybraną mechaniką było dodanie do MIMUW Surfers:
 - Mechaniki zdobywania monet
- Umieszczamy stałą liczbę monet w równych odległościach. Dla każdej monety losujemy tor, na którym się znajdzie.
- Umieszczamy monetę wysoko, a następnie obniżamy ją, aby wykryć kolizję z potencjalną przeszkodą.

```
16
17 void ABaseLevel::PlaceCoin(FVector LevelBounds, const float Step, int i)
18 {
19     const int Lane = UKismetMathLibrary::RandomIntegerInRange(Min: 0, Max: Lanes.Num() - 1);
20
21     // Raise the Z value (e.g., +1000) so the trace starts ABOVE the train.
22     FVector Location(InX: GetActorLocation().X + LevelBounds.X - 1 * Step, InY: GetActorLocation().Y + Lanes[Lane], InZ: Get
23
24     FHitResult Hit;
25     FVector Start = Location;
26
27     // Increase the trace length (e.g., 2000) so it reaches the floor from the new height.
28     FVector End = Location - FVector(InX: 0, InY: 0, InZ: 2000.f);
29
30     FCollisionQueryParams TraceParams;
31     TraceParams.AddIgnoredActor(this);
32
33     if (GetWorld()->LineTraceSingleByChannel((Hit, Start, End, ECC_Visibility, TraceParams))
34     {
35         // Use the exact hit location (whether it's the train roof or the floor)
36         float CoinZ = Hit.Location.Z + CoinOffset;
37
38         FVector CoinLocation = FVector(Hit.Location.X, Hit.Location.Y, CoinZ);
39
40         FActorSpawnParameters SpawnParams;
41         SpawnParams.SpawnCollisionHandlingOverride = ESpawnActorCollisionHandlingMethod::AdjustIfPossibleButAlwaysSpawn;
42
43         if (ACoin* SpawnedCoin = GetWorld()->SpawnActor<ACoin>(CoinClass, CoinLocation, Rotation: FRotator::ZeroRotator, Sp
44         {
45             SpawnedCoin->AttachToActor(this, FAttachmentTransformRules::KeepWorldTransform);
46         }
47     }
48 }
```



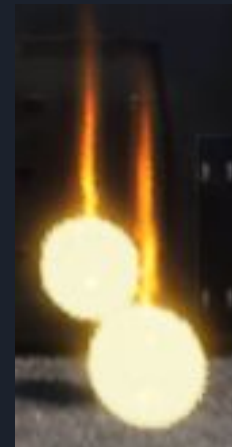
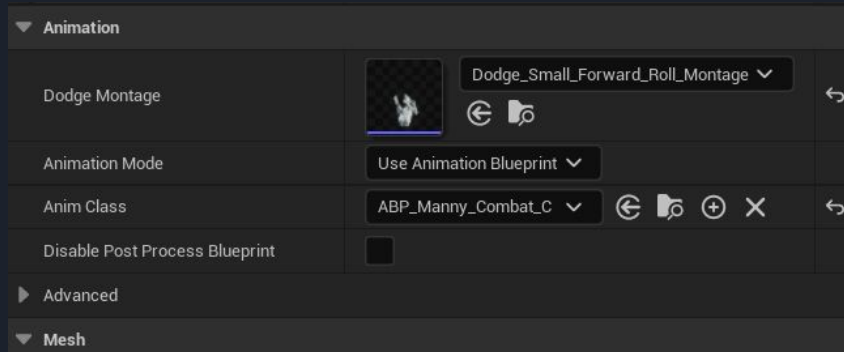
Wybrana mechanika

- Naszą wybraną mechaniką było dodanie do MIMUW Surfers:
 - Mechaniki zdobywania monet
 - Mechaniki boosterów - x2 score oraz 1.5 jump boost
- Boostery zaimplementowane w C++ i na podstawie klasy zrobione jako Blueprinty (obiekty wrzucone na mapę)
- Boostery mają swoją jedną klasę i Enum, dzięki czemu dodanie kolejnych jest proste i wygodne, podczas Overlapu boostera z graczem, na gracza nakładany jest efekt danego boostera



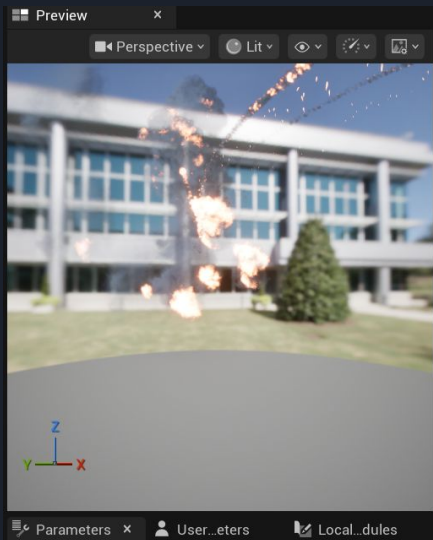
Animacje

- Gracz ma przypisaną animację przy turlaniu się pod przeszkodami, reszta została pozostawiona z bazowego manekina
- Podczas animacji turlania się, wielkość kolizji zmniejsza się, żeby nie walnąć głową która nie istnieje w jakąś przeszkodę na drodze
- Implementacja wyświetlania animacji w C++, w UE jedynie przypisanie określonej animacji do istniejącej wartości
- Coiny ładnie się świecą i obracają



Efekty cząsteczkowe

- Darmowe efekty pobrane z Unreal Fab:
 - Niagara explosion - przy przegranej gracza
 - Niagara loot effect - monety



Zaawansowane materiały - dźwięki

- Coiny odtwarzają dźwięk przy podniesieniu przez gracza
- Przy śmierci (zderzeniu z obiektem jest dźwięk porażki oraz wybuch - pokażemy przy okazji LIVE albo przy okazji sekcji o Interaktywnym UI)
- Podczas całej rozgrywki przygrywa nam muzyczka z Subway Surfers
- Odtwarzanie muzyczki zrobione w C++ podczas Overlapu obiektu z graczem, z wykorzystaniem obiektów Sound Cue

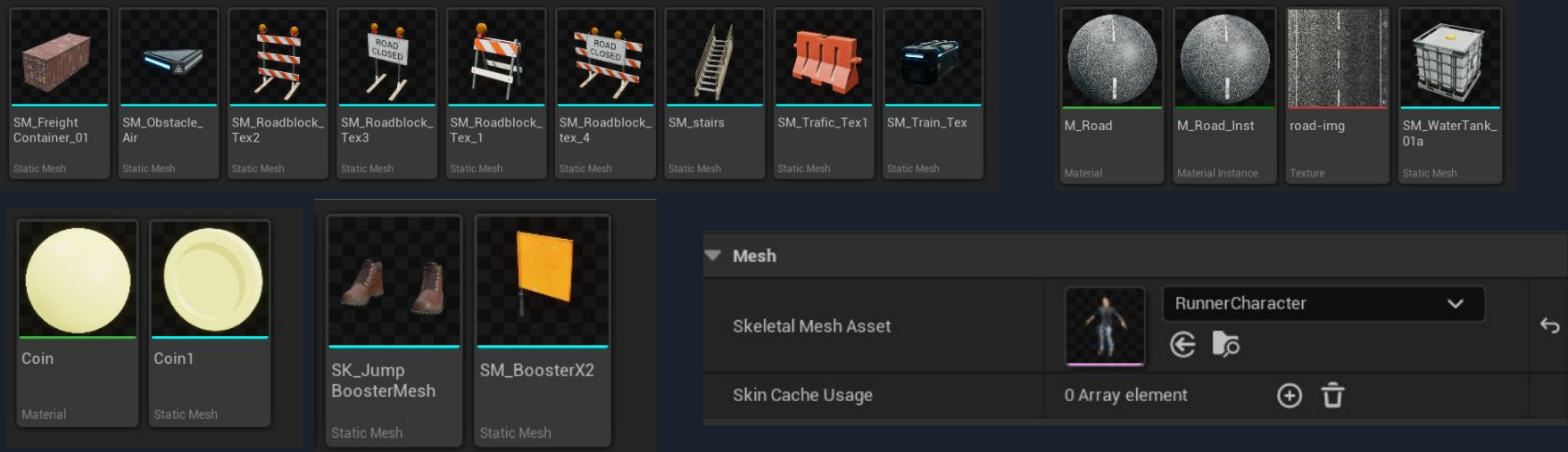


```
if (Spike || Train)
{
    UE_LOG(LogTemp, Warning, TEXT(InFormat: "Hit an obstacle! Restarting..."));

    // Play hit sound if assigned
    if (HitObstacleSound)
    {
        UGameplayStatics::PlaySoundAtLocation(WorldContextObject: this, HitObstacleSound, GetActorLocation());
    }
}
```

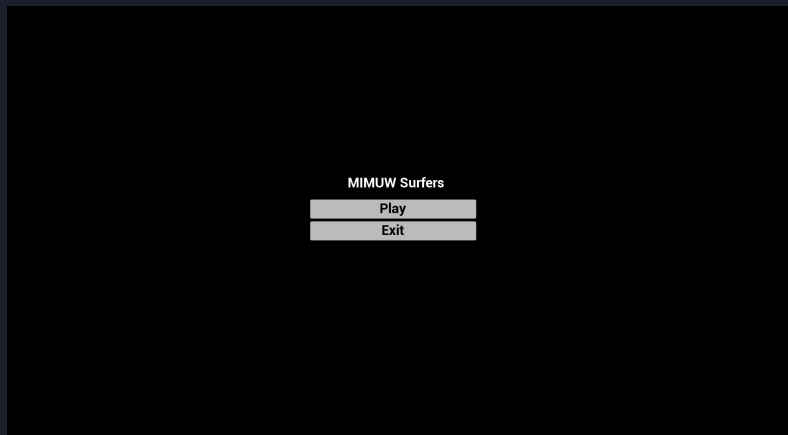
Zaawansowane materiały - tekstury

- Grafiki do modelu gracza, przeszkód, schodów, boosterów, coinów, drogi zostały zgrane z internetu, bądź ściągnięte z Fab'a w Epic Games Launcherze
- Przeszkody to obiekty Blueprinty z implementacją w C++, które mają określone StaticMeshe jako teksturki



Interaktywne UI

- W menu głównym możliwości:
 - Play -> rozpoczęcie gry
 - Exit -> zamknięcie gry
- Po śmierci gracza możliwość kliknięcia Retry, aby zacząć grę od nowa



MainMenu zrobione jako oddzielny Level z przyciskami, które OnClick przenoszą do gry/zamykają grę

```
if (GameOverWidgetClass)
{
    // 1. Create the Widget
    UUserWidget* GameOverWidget = CreateWidget<UserWidget>(OwningObject: GetWorld(), GameOverWidgetClass);

    if (GameOverWidget)
    {
        // 2. Add to Screen
        GameOverWidget->AddToViewport();

        // 3. Enable Mouse Cursor so player can click "Restart"
        APlayerController* PC = Cast<APlayerController>(GetController());
        if (PC)
        {
            PC->bShowMouseCursor = true;

            // Input Mode: UI Only (Stops player from moving character with keys)
            InputModeUIOnly InputMode;
            InputMode.SetWidgetToFocus(GameOverWidget->TakeWidget());
            PC->SetInputMode(InputMode);
        }
    }
}
```

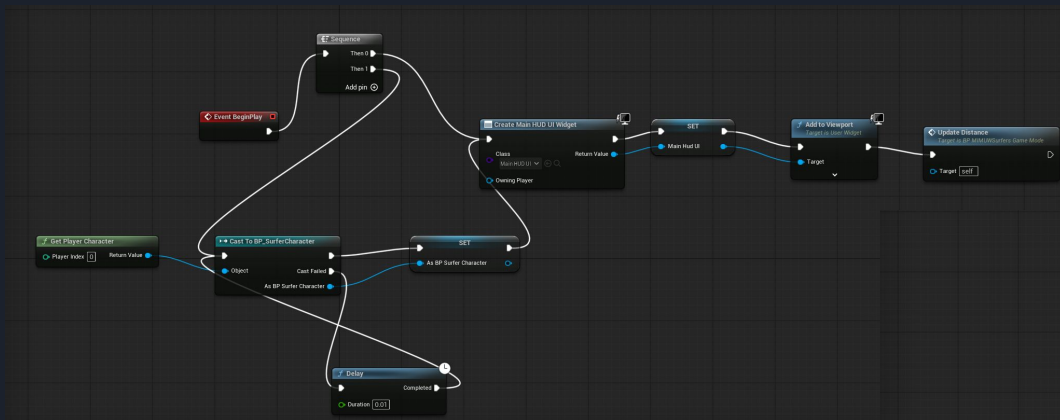
Wyświetlanie Game Over po trafieniu na Obstacle

Interaktywne UI

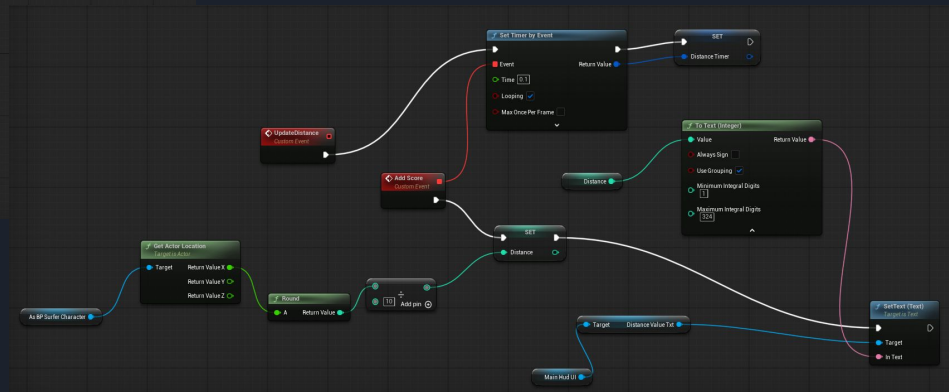
- Podczas rozgrywki wyświetlane jest aktywnie:
 - Liczba zebranych przez gracza monet
 - Długość przebytego dystansu

Coins: 5

Distance: 245



Aktywny update przebytego dystansu przez gracza w blueprintach



Interaktywne UI

- Gra zapisuje High Score po porażce -> ilość zdobytych monet oraz przebyty dystans
- Po zderzeniu gracza z obiektem, który może go zabić, wywoływana jest metoda C++ SaveGame(), która zapisuje stan wyniku oraz liczby monet

```
void ASurferCharacter::SaveGame()
{
    // Try to LOAD the existing save file first
    USurferSaveGame* SaveInst = Cast<USurferSaveGame>({
        @: UGameplayStatics::LoadGameFromSlot(SaveSlotName, UIndex 0));

    // If it doesn't exist, create a new one
    if (!SaveInst)
    {
        SaveInst = Cast<USurferSaveGame>({
            @: UGameplayStatics::CreateSaveGameObject(USurferSaveGame::StaticClass());
        });

        // Update High Score (Independently)
        // We check if current Score is better than the Saved Score
        if (Score > SaveInst->HighScore)
        {
            SaveInst->HighScore = Score;
            // Update local variable so the Widget sees the new high score immediately
            HighScore = Score;
        }
        else
        {
            // If we didn't beat it, ensure our local variable matches the best saved one
            HighScore = SaveInst->HighScore;
        }

        // Update Highest Coins (Independently)
        // We check if current Coins are better than Saved Coins
        if (Coins > SaveInst->HighestCoins)
        {
            SaveInst->HighestCoins = Coins;
            HighestCoins = Coins;
        }
        else
        {
            HighestCoins = SaveInst->HighestCoins;
        }

        // Write the final result to disk
        UGameplayStatics::SaveGameToSlot(SaveInst, SaveSlotName, UIndex 0);

        UE_LOG(LogTemp, Warning, TEXT("Game Saved! High Score: %f, Best Coins: %d"), HighScore, HighestCoins);
    }
}
```

- Po zapisie wyświetlany jest widget, który był na poprzednim slajdzie, który zczytuje dane zapisane przez tę funkcję



Dzięki!

Pytania?

Agnieszka Suszko & Konrad Mocarski